PROJECT MILESTONE 4 REPORT | CSE MACRO PROJECT

RAGTAG: Production Ingestion Pipeline, Conversational Disambiguation, & Agentic Workspace

Complete Technical Report on Desktop ZIP Ingestion, Multi-Turn Context Intelligence, Priority Rules, and 4-Tier Verification

ACADEMIC TRACK: 3rd Sem Macro Project (CSE)	**PROJECT REPO: Team RAGTAG Workspace**
DELIVERABLE: Milestone 4 Technical Report	**VERIFICATION:** 41 / 41 Automated Tests Passed (100%)
ACTIVE USER CONTEXT: Guru (settings.json)	**CORE STACK:** Next.js 16 | SQLite Ground Truth | Ollama Llama-3 | ChromaDB

1. Executive Summary & Milestone Objective

Following the initial relational schema established in prior milestones, **Milestone 4** represents the full operational realization of the offline, privacy-first RAGTAG platform. This phase completely supersedes synthetic and simulated mockups with 100% authentic WhatsApp data flows. Critical engineering advancements delivered include: (1) Native WhatsApp Desktop ZIP and TXT ingestion with deterministic deduplication and group isolation; (2) Context-aware conversational thread resolution utilizing a [-20 min, +5 min] dialogue window and burst inheritance; (3) Strict grammatical disambiguation via first-person pronoun rules and personal user filtering; (4) An algorithmic priority scoring engine based on temporal thresholds and authoritative directives; (5) An interactive dual-pane WhatsApp console with direct context landing and visual pulsing markers; (6) One-click Google Calendar URL generation; (7) Dual-engine extraction pairing local Ollama Llama-3 with instant heuristic failover; and (8) A rigid 4-tier automated test harness with 41 passing test cases (100% pass rate).

2. WhatsApp Desktop ZIP Ingestion & Group Separation

Native Compressed ZIP & TXT Ingestion Pipeline (POST /api/upload):

WhatsApp Desktop packages exported chat logs into compressed .zip containers containing chat logs and media references. The backend ingestion engine in backend/ingest.py was upgraded to accept both compressed ZIP archives and raw TXT files. Archives are uncompressed in-memory, validated, and normalized into SQLite ground truth without manual pre-processing.

Automatic Group Naming & Channel Namespace Isolation:

Previously, multi-group imports collapsed distinct conversations into a monolithic chat stream. The engine now dynamically extracts the exact channel namespace from the archive's filename (e.g., Aerospace Members 2026.zip automatically instantiates and scopes all messages to the 'Aerospace Members 2026' relational chat channel), ensuring clean separation across academic and personal groups.

Idempotent Deduplication via Deterministic Message Hashing:

To support recurring, incremental imports without data redundancy, every message is hashed (SHA-256 over timestamp, sender JID, channel namespace, and sanitized body text). Re-importing an updated archive days later appends only genuinely new messages, enforcing 100% idempotency and eliminating duplicate records and wasted storage.

Real-Time Upload Loading Bar & 4-Stage Ingestion Stepper:

The frontend /sync interface was enhanced with a real-time byte-level upload progress bar (0% -> 100%) and a visual 4-stage pipeline stepper: **Stage 1: Upload ZIP** -> **Stage 2: SQLite Relational Ingest** -> **Stage 3: Task Extraction Engine** -> **Stage 4: ChromaDB Dense Vector Embeddings**.

Stage 1: File Transport	Stage 2: Relational Ground Truth	Stage 3: Task Intelligence	Stage 4: Vector Indexing
Multipart byte stream (0% -> 100%) to /api/upload	SQLite parsing, Foreign Key validation & SHA-256 deduplication	Ollama Llama-3 extraction with 1.0s heuristic fallback	ChromaDB 384-dim ONNX MiniLM vector indexing for RAG

3. Conversational Thread Resolution & 'Is It Me?' Intelligence

Natural group chats are fragmented: participants rarely mention explicit names in every instruction. Milestone 4 implements a dedicated multi-turn conversational resolution layer that inspects conversational context to accurately determine task authorship, recipient identity, and personal relevance.

- **Temporal Context Window [-20m, +5m]:** The resolution engine examines a 20-minute preceding and 5-minute succeeding dialogue window surrounding every candidate directive. *Case Study:* When Mahima posts *'Okay then send me the report ASAP!!!'*, the engine traces prior turns, identifies Lakshyyy as the active respondent who initiated the exchange, and correctly assigns the task to Lakshyyy rather than Guru.
- **Same-Sender Burst Inheritance:** When a group member tags someone in an opening query (e.g., Shreya: *'@~Tanishk attendance??'*) and immediately follows with instructional bursts (*'Write branch and year also.. and check spellings'*), the follow-up directives automatically inherit Tanishk as the target assignee.
- **Strict 1st-Person Pronoun Enforcement:** Grammatical parsing strictly mandates that directives containing 'me', 'my', or 'send to me' assign the requester role to the author of that message (e.g., Mahima or Shreya). The task is never misattributed to Guru unless Guru authored the message.
- **WhatsApp Pushname Sanitization:** Raw WhatsApp exports include mobile pushname prefixes (e.g., @~Lakshyyy or ~Tanishk). The preprocessor cleanses these into canonical identifier tokens (Lakshyyy, Tanishk) before entity matching.
- **The 'Is It Me?' Verification Filter:** Candidate assignees are normalized, stripped of emojis, and evaluated against settings.json (configured with user_name: Guru). Tasks assigned to external team members remain indexed for team visibility but are filtered out from Guru's personal action board.

> **Architectural Impact:** By combining temporal dialogue lookback with grammatical pronoun binding, false-positive task assignment to the primary user dropped to 0% across all evaluated multi-member academic group archives.

4. Deterministic Task Prioritization Engine

Random or heuristic-based priority tags have been replaced with a deterministic, multi-variable rule matrix:

Priority Level	Trigger Conditions & Algorithmic Rules	Visual UI Designation
HIGH PRIORITY	• Explicit urgency triggers: asap, urgent, without fail, mandatory • Hard deadlines scheduled within **<= 48 hours** • Directives issued by verified evaluators, mentors, or professors	**Red Accent Badge** High-visibility banner
MEDIUM PRIORITY	• Standard academic milestones and scheduled deliverables • Action items with deadlines occurring within **<= 7 days**	**Amber Accent Badge** Standard task card
LOW PRIORITY	• Informational tasks, reading recommendations, or open-ended items without hard deadlines	**Slate Accent Badge** Subtle card style

5. Direct Google Calendar Integration (1-Click 'APPROVE + REMIND')

Previously, calendar integration required downloading an RFC-5545 .ics file and manually importing it into an external client. Milestone 4 replaces this with a direct **'APPROVE + REMIND'** button on every task item. When clicked, the application automatically generates and launches a formatted Google Calendar template URL (https://calendar.google.com/calendar/render?action=TEMPLATE...). The URL pre-fills the task title, ISO due date, reminder notifications, and the verified WhatsApp chat quote as reference context.

6. Interactive WhatsApp-Like Workspace & Deep Context Landing

2-Pane WhatsApp Console:

A desktop-first two-pane console emulates the intuitive layout of WhatsApp Web: the left pane renders group channels and contacts with active unread badges and snippet previews, while the right pane renders the chronological dialogue stream and AI briefings.

Direct Context Landing ('Open in Agentic Panel'):

To provide total verification transparency, task cards feature a *'View Context'* button and audit modals feature an *'Open in Agentic Panel'* action. Clicking it instantly jumps to the corresponding chat channel, automatically scrolls the exact origin message into the viewport, and highlights it with a **pulsing amber ring and badge**, allowing users to immediately review surrounding conversational context.

7. Ground-Truth Migration & Dual-Engine Intelligence

Purging Synthetic Mockups for Ground-Truth Data:

All legacy 'vibe-coded' simulation triggers and fake faculty buttons have been permanently purged from the codebase. The platform now operates exclusively on real WhatsApp exports, authentic sender metadata, and verified SQLite records.

Dual-Engine Architecture (Ollama + Deterministic Heuristics):

The task extractor pairs local llama3 via Ollama (for deep contextual and temporal extraction) with a 1.0-second circuit-breaker timeout that automatically fails over to 100% offline deterministic heuristics if the LLM is busy or unavailable.

Catch-Me-Up Briefings & ChromaDB Semantic RAG:

The briefing engine condenses 100+ unread messages into structured categories: *Key Decisions*, *Action Items*, and *Urgent Mentions* across customizable presets (2 hours, 24 hours, 7 days). Complementing briefings, the POST /api/chat endpoint conducts vector semantic retrieval over local ChromaDB embeddings, returning grounded chat citations with exact message IDs.

8. Quality Assurance: 4-Tier Automated Test Harness (41/41 Passed)

All subsystems undergo verification via the project's automated test runner (run_tests.py), achieving a **100% pass rate**:

Tier	Verification Scope & Targeted Subsystems	Tests	Pass Rate	Status
Tier 1	Unit & Schema: SQLite ORM models, mandatory foreign keys, trigger rules, date parsing	17	100%	**PASSED (17/17)**
Tier 2	Ingestion & Extractor: ZIP/TXT parsing, deduplication, dual-mode fallback, upgrades	11	100%	**PASSED (11/11)**
Tier 3	Flask API Contracts: Upload, task mutations, briefings, settings, and vector chat	11	100%	**PASSED (11/11)**
Tier 4	E2E Simulation & Air-Gap: End-to-end task auditability, zero external network leakage	2	100%	**PASSED (2/2)**
TOTAL	**Complete System Engineering Verification**	**41**	**100%**	**41 / 41 PASSED (100%)**

9. Milestone 4 Conclusion & Phase 2 Outlook

Conclusion: Milestone 4 brings Team RAGTAG from initial database design to an end-to-end operational intelligence platform. The system achieves 100% offline data integrity, zero cloud dependency, robust conversational disambiguation, and verified reliability.
Phase 2 Outlook: As documented in FUTURE_PLANS.md, upcoming efforts will package the Next.js frontend and Flask backend into a standalone desktop application executable (.exe) utilizing **Electron** or **Tauri**, eliminating the need for terminal commands or a web browser.